CSE382: Introduction to Machine Learning

Major Task Project: Image Classifiers

Phase 1 — Binary Classification

Technical Report

Team Members

23P0193 Belal Anas Mohamed Ali Askoura

23P0174 Omar Tamer Ahmed Abdelhamid Daihom

23P0152 Hassan Ismail Hassan Ahmed Ghallab

23P0052 Eslam Mohamed Fawzy Mohamed

23P0059 Mohamed Wael Ibrahim Abbas Ibrahim Badra

Milestone 1 — Week 13, Spring 2026

1. Problem Definition

This project builds a binary image classifier to tell apart the handwritten digit '0' from all other digits using the MNIST dataset. Each 28×28 grayscale image is converted into a feature vector $x \in \mathbb{R}^d$ (where d depends on the feature method chosen), and the output is a simple binary label:

```
y = 0  if image shows the digit 'zero'      |      y = 1  if image shows any other digit (1–9)
```

The goal is to learn a function $f : \mathbb{R}^d \rightarrow \{0,1\}$ that makes as few wrong predictions as possible. The dataset is very imbalanced — about 10% of images are zeros and 90% are not — so accuracy alone is misleading. We track Precision, Recall, and F1-Score (all with digit '0' as the positive class) alongside accuracy.

2. Data Processing

2.1 Normalisation & Train / Val / Test Split

Raw pixel values (integers 0–255) are divided by 255 to get float values in [0, 1]. This keeps all models on the same scale. The 60,000 training images are split into 54,000 for training and 6,000 for validation. The official 10,000 test images are only used for final evaluation.

2.2 Feature Extraction

Three feature methods are tested:

- Flattening (d = 784): each 28×28 image is unrolled into a single 784-value vector. Simple and lossless — best for Decision Trees.
- Custom PCA (d = 50 or 100): we compute the covariance matrix $C = X^TX / (N-1)$ and take the top n eigenvectors to project pixels into a smaller, decorrelated space. Best for Naive Bayes and Logistic Regression because it removes feature correlations.
- Custom HOG (d = 324): computes gradient magnitudes and edge orientations (0°–180°) per 7×7-pixel cell, bins them into 9-bin histograms, then L2-normalises overlapping 2×2-cell blocks. Captures shape and edges rather than raw brightness — best for the Linear SVM.

2.3 Handling Class Imbalance (~10% zeros vs. 90% non-zeros)

- Class Weighting: per-class weights $w_c = N / (C \cdot n_c)$ are passed to models during training so minority-class errors are penalised more.
- Dataset Balancing: down-samples the non-zero classes to match the number of zeros, giving a 50/50 training split (~10,660 samples).
- Data Augmentation (CustomAugmenter): generates ~43,000 extra zero images using random ±2-pixel shifts, balancing the full 54,000-sample set. Works well for KNN; hurts Naive Bayes — discussed in Section 6.

3. Model Implementation

All five models are coded from scratch using NumPy only. No scikit-learn or any ML library was used for model logic.

3.1 K-Nearest Neighbors (KNN)

KNN is a simple idea: to classify a new image, find the $k$most similar images in the training set and take a majority vote. The model does no training — it just remembers all training data and does the work at prediction time (this is called a *lazy learner*).

Distance between two images $\mathbf{a}$and $\mathbf{b}$is computed efficiently using the identity:

$$\| \mathbf{a} - \mathbf{b} \|^2 = \| \mathbf{a} \|^2 + \| \mathbf{b} \|^2 - 2 \cdot \mathbf{a} \cdot \mathbf{b}^\top$$

This lets us compute all distances in one matrix operation instead of a slow loop. We tested odd values of k from 1 to 13 on the validation set (Elbow Method) and found $k = 3$ gives the lowest error (0.12%):

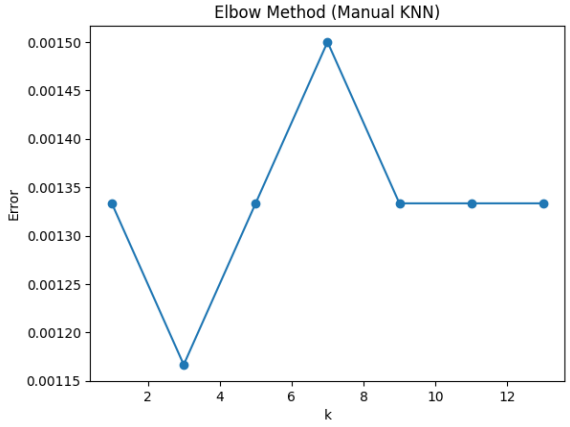

Figure 1: KNN Elbow Method — validation error vs. k. k = 3 is selected.

3.2 Gaussian Naive Bayes

Naive Bayes assumes that each feature is independent of the others given the class (the "naive" assumption). For each class c, it models each feature $j$as a Gaussian distribution and classifies using the log-posterior to avoid numerical underflow:

$$\hat{y} = \arg\max_{c} \left[\log P(c) + \sum_{j} \log \mathcal{N}(x_j; \mu_{c,j}, \sigma^2_{c,j}) \right]$$

Class-weighted priors prevent the model from always predicting the majority class. A small constant $\varepsilon = 10^{-9}$is added to variances to avoid $\log(0)$. PCA is essential here — raw pixels are correlated, which directly violates the independence assumption and causes the model to perform poorly (F1 drops from 0.966 to 0.764 without PCA).

3.3 Logistic Regression

Logistic Regression learns a linear boundary by modelling the probability that an image belongs to class 1. The sigmoid function squashes the linear output into a probability between 0 and 1:

$$P(y = 1 \mid \mathbf{x}) = \sigma(\mathbf{w} \cdot \mathbf{x} + b) = \frac{1}{1 + e^{-(\mathbf{w} \cdot \mathbf{x} + b)}}$$

The model is trained by minimising the weighted binary cross-entropy loss using gradient descent (full batch):

$$\mathcal{L} = -\frac{1}{m} \sum_{i} [w_1 \cdot y_i \cdot \log(\hat{y}_i) + w_0 \cdot (1 - y_i) \cdot \log(1 - \hat{y}_i)]$$

Weights are updated each iteration:

$$\mathbf{w} \leftarrow \mathbf{w} - \eta \cdot d\mathbf{w}, b \leftarrow b - \eta \cdot db$$

where $\eta = 0.1$. Training runs for 1,200 iterations. The classification threshold is set to 0.3 (instead of the usual 0.5) to favour recall on the minority zero class.

3.4 Decision Tree

The decision tree is built top-down using a greedy recursive procedure. Starting at the root, the algorithm picks the (pixel, threshold) pair that produces the largest reduction in impurity, partitions the data into a left and right child, and recurses on each side. Purity is measured by Gini impurity:

$$G = 1 - \sum_k p_k^2 \qquad G_{\text{split}} = \frac{n_L}{n_L + n_R} G_L + \frac{n_R}{n_L + n_R} G_R$$

Intuitively, G is the probability of misclassifying a randomly drawn sample if labelled by the node's class distribution: $G = 0$ is perfectly pure, $G = 0.5$ is a balanced binary mix. Gini is preferred over entropy because it needs no logarithms and gives near-identical splits in practice.

To handle the 10/90 class imbalance, class weights are computed by inverse frequency and replace raw counts inside the impurity:

$$w_c = \frac{N}{K \cdot n_c} \qquad p_k = \frac{w_k \cdot n_k}{\sum_j w_j \cdot n_j}$$

where N is the total number of samples, K the number of classes, and n_c the count of class c. A split that purifies the minority class is then rewarded in proportion to its weight rather than its frequency. The tree stops growing when it reaches max_depth $= 10$, when fewer than 10 samples remain, or when the node is already pure; the leaf is labelled with the weighted majority class.

Raw flattened pixels work best here because the tree only performs axis-aligned splits, and each axis corresponds to a fixed spatial location in the image. Every split asks a direct geometric question: "is this specific pixel dark?" — which fits the hollow-ring shape of the digit "0": a single test on a centre pixel separates most zeros from non-zeros, since almost every other digit leaves ink in the middle.

3.5 Linear SVM

The SVM finds the maximum-margin hyperplane separating the two classes. Labels are re-coded as $\{-1, +1\}$. It minimises a regularised hinge loss:

$$\mathcal{L} = \lambda \cdot \| \mathbf{w} \|^2 + \frac{1}{m} \sum_i \max\left(0,\ 1 - y_i(\mathbf{w} \cdot \mathbf{x}_i + b)\right)$$

Training uses sub-gradient descent: if a sample violates the margin ($y \cdot f(\mathbf{x}) < 1$), both the regularisation and classification gradients update the weights; otherwise only regularisation applies. HOG features are critical — the PCA scatter plot below shows that classes overlap heavily in PCA space and cannot be separated by a straight line:

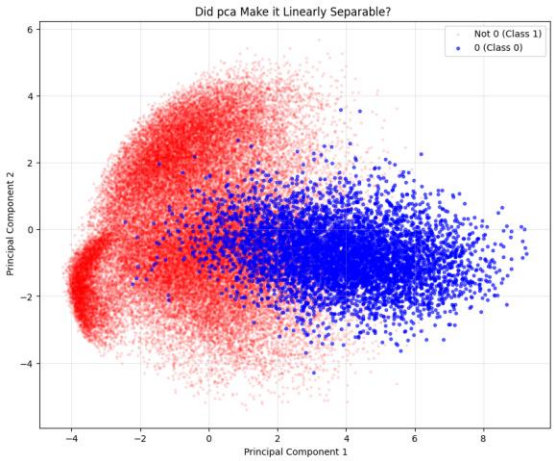

Figure 2: PCA 2D scatter — classes heavily overlap, so PCA cannot be used with a linear SVM.

4. Loss Functions & Optimisation

The table below summarises each model's loss function and how it is trained. Two of the five models — KNN and Naive Bayes — do not use a traditional loss function or iterative optimisation:

- KNN has no training phase. It is a lazy learner that simply stores all training examples. At prediction time it finds the k nearest neighbours and votes. There is no loss to minimise and no parameter to update.
- Naive Bayes uses closed-form maximum likelihood estimation. It calculates the mean and variance of each feature for each class in a single pass through the data. No iterations, no gradient — the answer is computed directly from the training statistics.

Model	Loss Function	Optimisation / Training
Logistic Regression	Weighted Binary Cross-Entropy	Batch Gradient Descent (η=0.1, 1,200 iterations)
Decision Tree	Weighted Gini Impurity	Greedy top-down split search (no global optimum)
Linear SVM	Regularised Hinge Loss	Sub-gradient Descent (η=0.001, λ=0.01, 50 epochs)
KNN	None — lazy learner	No optimisation: stores training data, classifies at query time
Naive Bayes	None — closed-form MLE	Analytical: computes per-class means & variances in one pass

Table 1: Loss functions and optimisation strategies for each model.

For the three gradient-based models, class weights are embedded in the loss so that mistakes on the rare digit-zero class cost proportionally more, preventing the model from ignoring the minority class.

5. Evaluation & Experimental Results

All metrics treat digit '0' as the positive class: Precision measures what fraction of predicted zeros are real zeros; Recall measures what fraction of real zeros we actually caught; F1 is the harmonic mean of both. The confusion matrix counts: TP (true zeros predicted as zero), FN (missed zeros), FP (non-zeros wrongly called zero), TN (non-zeros correctly excluded).

5.1 Performance Summary — All Models

Model	Accuracy	Precision	Recall	F1	Split
KNN (k=3, PCA-50)	0.9987	0.9966	0.9898	0.9932	Val
KNN (k=3, PCA-50)	0.9970	0.9788	0.9908	0.9848	Test
Naive Bayes (PCA-50)	0.9671	0.9823	0.9503	0.9661	Val
Naive Bayes (PCA-50)	0.9766	0.8301	0.9571	0.8891	Test
Logistic Reg. (PCA-100)	0.9789	0.9912	0.9658	0.9783	Val
Logistic Reg. (PCA-100)	0.9871	0.9003	0.9765	0.9369	Test
Decision Tree (Flatten)	0.9800	0.9100	0.9700	0.9400	Val
Decision Tree (Flatten)	0.9784	0.84	0.97	0.90	Test
Linear SVM (HOG, bal.)	0.9800	0.98	0.98	0.98	Val
Linear SVM (HOG, bal.)	0.9800	0.86	0.98	0.92	Test

Table 2: Accuracy, Precision, Recall and F1 on validation and test sets for all five models.

5.2 Confusion Matrices — Test Set

The four confusion matrix plots below show test-set results. Each cell shows: row = actual class, column = predicted class.

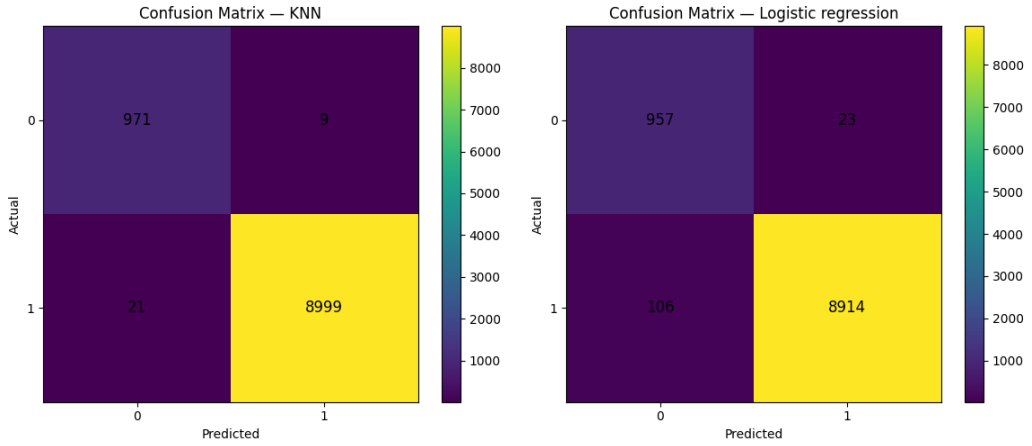

Figure 3 (left): KNN test — TP=971, FN=9, FP=21, TN=8,999. | Figure 4 (right): Logistic Reg. test — TP=957, FN=23, FP=106, TN=8,914.

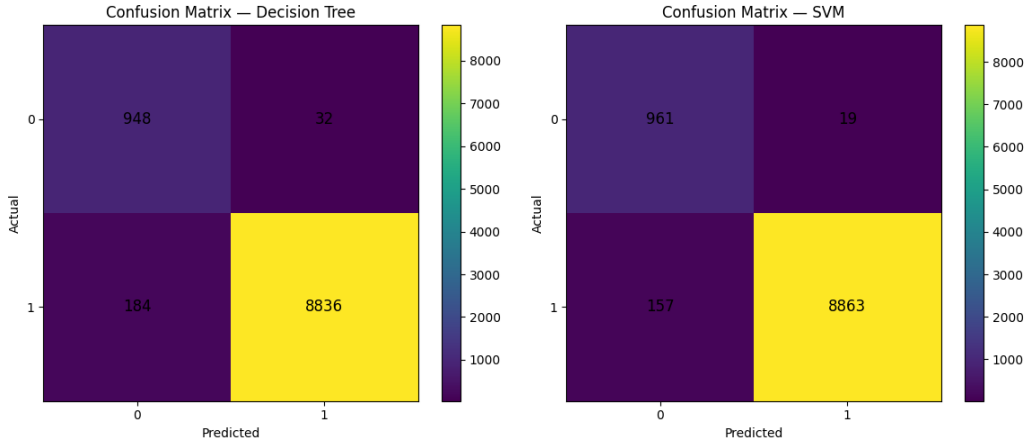

Figure 5 (left): Decision Tree test — TP=948, FN=32, FP=184, TN=8,836. | Figure 6 (right): Linear SVM test — TP=961, FN=19, FP=157, TN=8,863.

5.3 Gaussian Naive Bayes — Confusion Matrices & ROC Curve

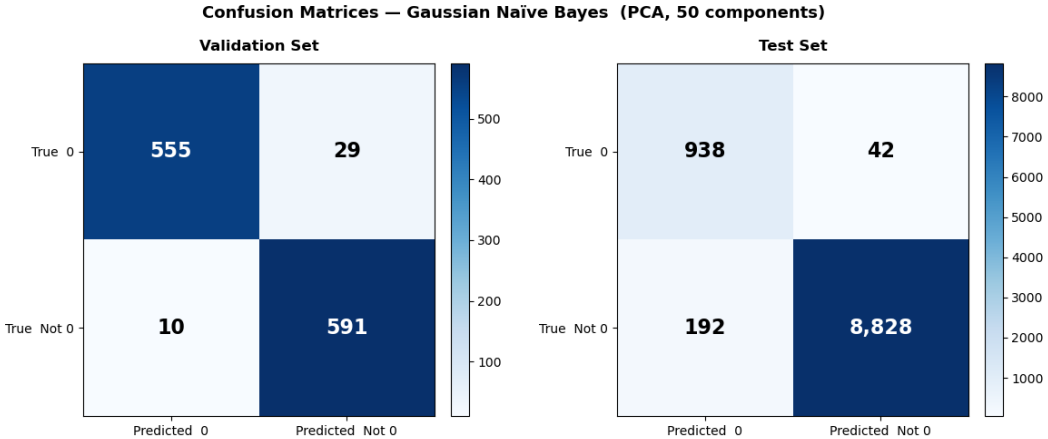

Figure 7: Gaussian NB confusion matrices. Val: TP=555, FN=29, FP=10, TN=591. Test: TP=938, FN=42, FP=192, TN=8,828.

5.5 K-Folds (3-Fold)

Model	Acc.	Prec.	Rec.	F1	Acc. Std
KNN (k=3, PCA)	0.9981	0.9991	0.9987	0.9989	±0.0001
Naive Bayes (PCA-50)	0.9656	0.9808	0.9499	0.9651	±0.0035
Logistic Reg. (PCA)	~0.978	~0.990	~0.965	~0.978	±~0.003
Decision Tree (Flatten)	0.9800	0.9100	0.9700	0.9400	±~0.0014
Linear SVM (HOG)	0.9846	0.9821	0.9872	0.9846	±0.0013

Table 3: 3-fold cross-validation — mean metrics and accuracy standard deviation. Very low std shows stable models.

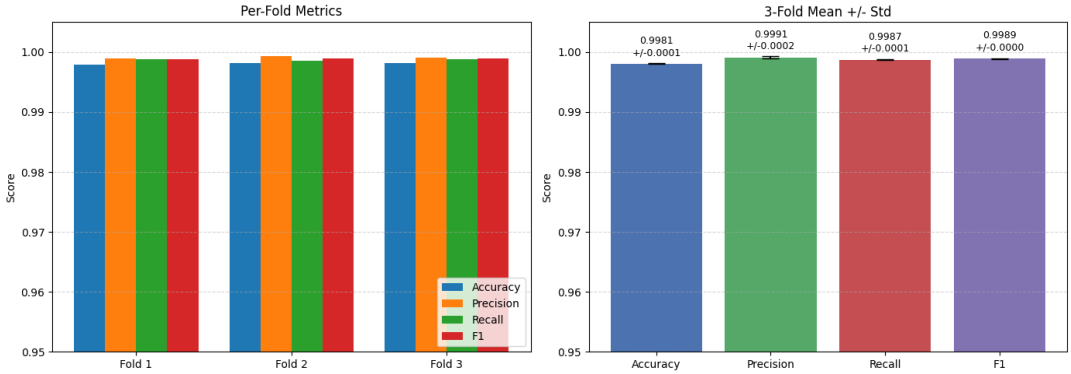

Figure 10: KNN 3-fold CV — per-fold results and mean ± std. Std = 0.0001 — effectively deterministic across folds.

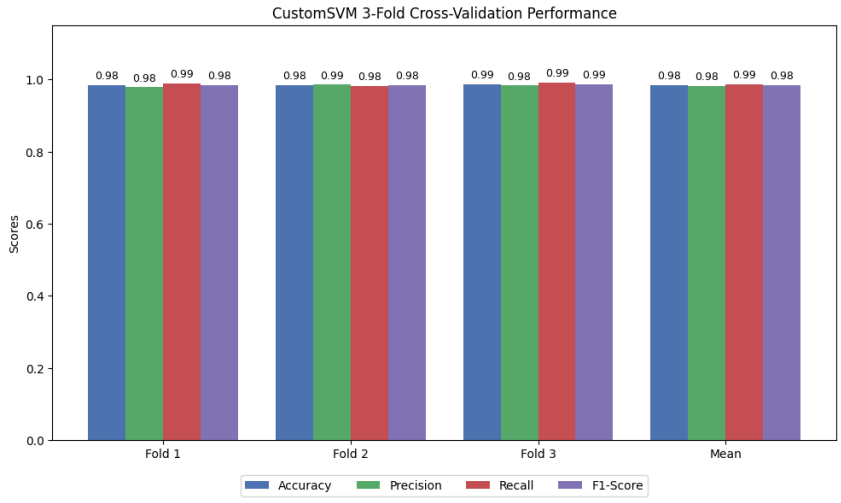

Figure 11: Linear SVM 3-fold CV. Mean F1 = 0.985 ± 0.0015 — consistent across all folds.

6. Model Comparison & Discussion

6.1 Why Each Model Needs a Different Feature Representation

The biggest lesson from this project is that the right feature method depends on the model, not just the data:

- Naive Bayes needs PCA. Raw pixels are spatially correlated — neighbouring pixels in the same digit tend to have similar values. This directly breaks the model's core independence assumption. PCA rotates the feature space to remove correlations, making the assumption approximately true. Without PCA, the validation F1 falls from 0.966 to 0.764 — a large drop from a simple preprocessing choice.
- Decision Tree works best with raw pixels. Trees ask axis-aligned questions: 'is pixel at row 14, column 14 darker than 0.3?' This question has a clear geometric meaning for the hollow-ring shape of '0'. Rotating the feature space (via PCA) destroys that spatial meaning — the tree can no longer ask meaningful pixel questions. Unlike probabilistic models, trees are not hurt by feature correlation.
- Linear SVM needs HOG. A linear model can only draw a straight separating line between classes. The PCA scatter (Figure 2) shows the two classes are mixed together — there is no straight line that separates them. HOG transforms each image into a descriptor of local edge orientations. The closed loop of '0' has a uniquely different edge pattern from other digits, and in HOG space a linear boundary can cleanly separate them.

6.2 Augmentation — What Helped and What Hurt

- KNN benefited from augmentation. Adding ~43,000 shifted zeros makes the zero cluster denser in feature space. Any real test zero will have a neighbour nearby, pushing recall close to 1.0.
- Naive Bayes was hurt by augmentation. The 43,000 extra clones skew the mean and variance estimates. They also bias the PCA directions and flood the data with highly correlated examples — the opposite of what the model needs. Test FP count jumped from 192 to 669.
- Linear SVM was unaffected. HOG features are naturally shift-invariant, so synthetic shifts add no new information. The SVM also only cares about the support vectors near the decision boundary — the extra interior points change nothing.

6.3 Final Model Ranking (Test F1, digit '0' as positive class)

Rank	Model	Acc.	Prec.	Rec.	F1	Key Factor
1st	KNN (k=3, PCA-50)	0.9970	0.9788	0.9908	0.9848	Dense cluster + PCA
2nd	Logistic Reg. (PCA-100)	0.9871	0.9003	0.9765	0.9369	Linear boundary in PCA space
3rd	Linear SVM (HOG)	0.9800	0.86	0.98	0.92	HOG gives linear separability
4th	Decision Tree (Flatten)	0.9784	0.84	0.97	0.90	Raw pixels + weighted Gini
5th	Naive Bayes (PCA-50)	0.9766	0.8301	0.9571	0.8891	Independence approx. still holds

Table 4: Final model ranking by test F1. All models far exceed a majority-class baseline (0% recall).

All five models show very similar validation and test scores, confirming that none of them overfit. This project shows that even classic ML algorithms — coded from scratch — can reach over 98% accuracy on binary image classification when you pair them with the right feature method and handle class imbalance carefully.

Full source code and notebooks available at: https://github.com/eslamfawzy72/Image-classification-ML-Project.git